

FIT3143 Applied 2

Julia Mallard, Nam Quang Tran

Task 1

Image Rotation with GPU

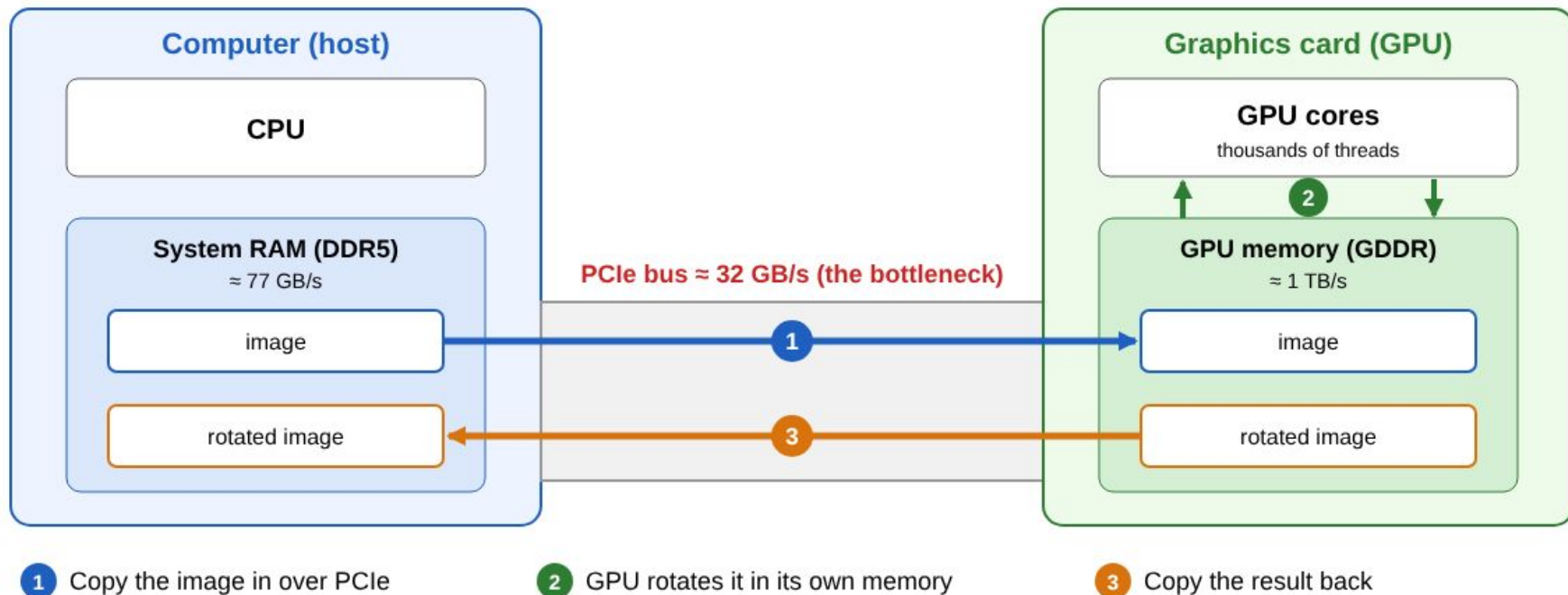
Moving image data from the CPU to the GPU

A discrete GPU has its own memory (GDDR), separate from the system's DDR5. The image must be copied across before the GPU can work on it, and the result copied back.

GPUs use GDDR for its much higher bandwidth (~ 1 TB/s vs ~ 77 GB/s), which feeds thousands of threads reading at once.

The GPU's memory system trades higher latency for that bandwidth, and hides the latency by switching between thousands of threads.

Moving the image to the GPU and back



PCIe is $\sim 30\times$ slower than GPU memory, so the copies take longer than the rotation.

Why not just...

- ...have the GPU read it from system memory?
 - The GPU would have to read it at 32 GB/s (PCIe speed) instead of 1 TB/s, about 30x slower.
 - Each read takes much longer, as it has to make a round trip across the bus.
 - It's better to make one big copy, rather than many smaller ones.
- ...keep everything in GPU memory?
 - You cannot start at the GPU, as the OS, the program and files are loaded into system memory.
 - Results would have to be sent back to the CPU anyway.
- ...use one memory for both?
 - My (Nam) laptop does that! I have a laptop with an M5 chip with 24 GB of unified memory.
 - The tradeoff is a lower peak bandwidth than a top discrete GPU (RTX 5090, PRO 6000, B200 etc), the memory bandwidth of an M5 chip is 153.6 GB/s compared to 1TB/s+ discrete GPUs can reach.
 - CPU and GPU would compete for the same memory and bandwidth.

How is data moved from CPU to the GPU?

- The GPU has its own copy engine (Direct Memory Access), a built-in mover that copies the image over PCIe (the connection to the GPU) while the CPU does other things.
- The copy engine needs the image to stay in one place in memory. Normal memory can be moved around by the OS, so the image is first copied into "pinned" memory, which is locked in place. Using pinned memory from the start skips this extra copy.
- The GPU rotates the image, and the copy engine sends the result back.

What is CUDA?

- CUDA is similar to OpenMP in that it splits work across threads, but it runs on the GPU and uses thousands of lightweight threads, each handling one small piece of data, such as one pixel. This is called data parallelism.
- OpenMP uses CPU threads, which share the CPU memory, however, CUDA threads run on the GPU's own memory, so it needs to be copied over.

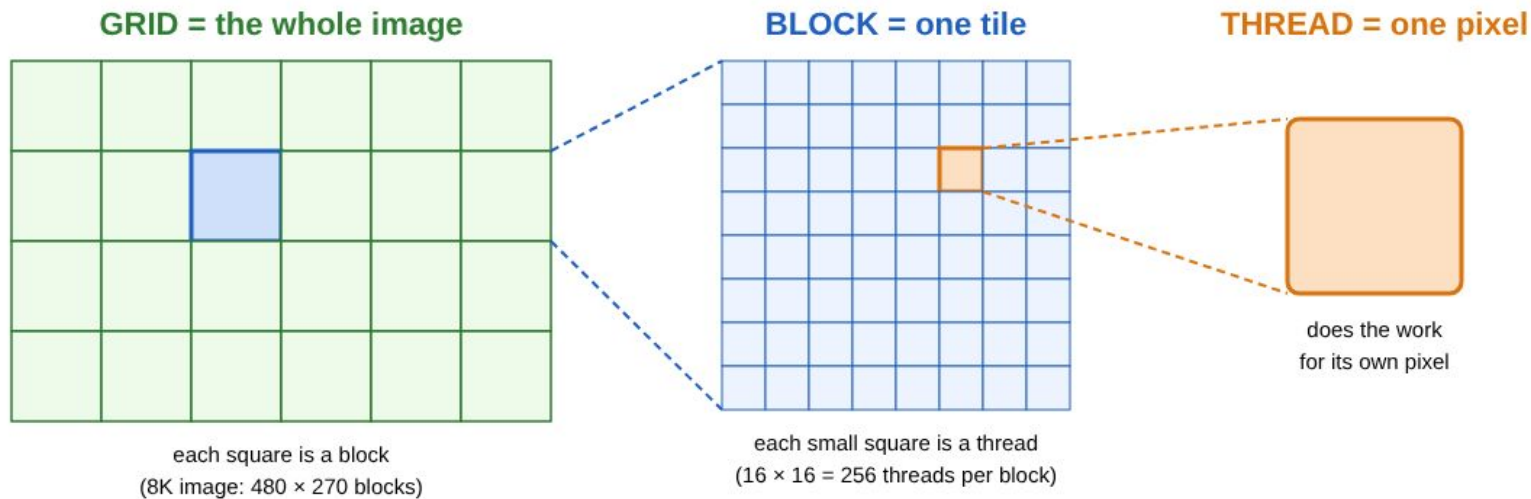
What the CPU and GPU are responsible for

Step	CPU (host)	GPU (device)
Prepare	Load the data (images) and decode it into RAM	
Reserve memory	Ask for space in GPU memory (cudaMalloc)	
Copy in	Start the copy to the GPU (cudaMemcpy)	The copy engine receives the image into GDDR
Launch	Choose the grid and block sizes and launch the kernel (rotate<<<grid, block>>>)	
Compute	Free to do other work (the launch doesn't wait)	Runs the kernel: millions of threads, one pixel each
Wait	Wait for the GPU to finish (cudaDeviceSynchronize)	
Copy out	Start the copy back (cudaMemcpy)	The copy engine sends the result to RAM
Clean up	Free GPU memory (cudaFree), then save or send the result	

What is a kernel?

- The kernel is the function the CUDA program runs on the GPU.
- When launched, it runs once per thread, so 33 million times for an 8K image.
- For the image rotation task, the GPU will go through this 33 million (or so) threads in waves, computing and moving onto more work.
- It is launched like this: `f<<<grid, block>>>`

Grid → Block → Thread



- Thread: the smallest unit of work. Each thread runs the kernel once, for one pixel.
- Block: a group of threads, here a 16×16 tile = 256 threads. A block runs on one SM, where its threads can share fast memory and work together.
- Grid: all the blocks from one kernel launch, covering the whole image. For 8K that's 480×270 blocks, shared out across the GPU's SMs.

```
__global__ void rotate(const uchar3 *src, uchar3 *dst, int w, int h, float c, float s) {  
    int x = blockIdx.x * blockDim.x + threadIdx.x;    // which pixel am I?  
    int y = blockIdx.y * blockDim.y + threadIdx.y;  
    if (x >= w || y >= h) return;  
    // inverse-rotate (x, y) to find the source pixel, then copy it  
}
```

```
dim3 block(16, 16); // 256 threads per block
```

```
dim3 grid((w + 15) / 16, (h + 15) / 16); // enough blocks to cover the image
```

```
rotate<<<grid, block>>>(d_src, d_dst, w, h, cosf(t), sinf(t));
```

Expected speedup using modern GPU architectures

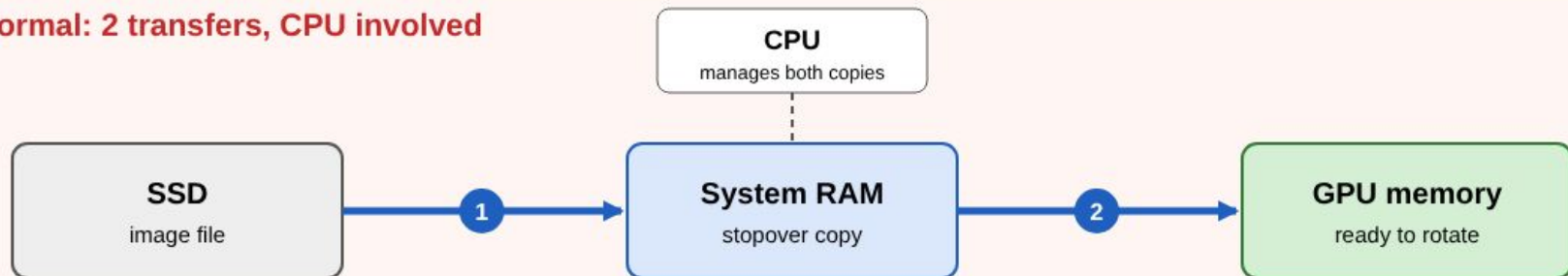
- Rotation is memory bound, as each pixel requires very little maths.
- The GPUs much higher peak memory bandwidth suits this problem well, meaning the speedup would be significant.
- I would expect to see about 5-15x speedup if rotating a single image, as both are limited by bandwidth, however, if looking holistically at the whole job, there would realistically only be 1-2x speedup, potentially none, as PCIe copying could take longer than the CPU working on the problem itself.

What is GPUDirect Storage?

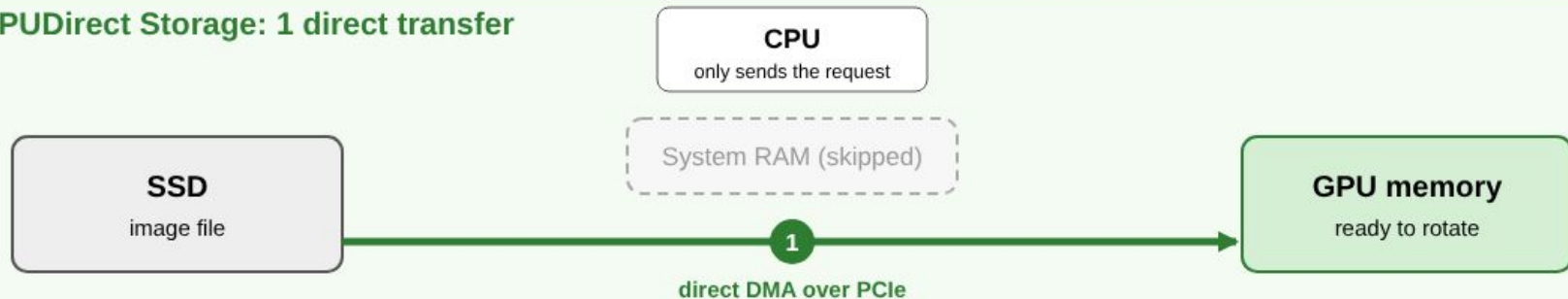
- Normally, files go persistent storage (SSD) → system RAM → GPU: two copies, managed by the CPU.
- GPUDirect Storage lets the SSD send data directly into GPU memory via DMA (direct memory access) (NVIDIA's cuFile)
- Less CPU work, higher bandwidth + lower latency
- Data still goes through PCIe, but skips the system RAM trip.
- It can only work on Linux systems.

Loading an image: normal path vs GPUDirect Storage

Normal: 2 transfers, CPU involved



GPUDirect Storage: 1 direct transfer



GDS removes the stop in system RAM: one transfer instead of two, and the CPU never handles the data.

(The data still crosses PCIe; it just goes straight from the SSD to the GPU.)

Would this assist in our photo rotation example?

- Helps: huge batches of large, uncompressed images; rotation is light, so data movement is the bottleneck.
- Doesn't help: compressed JPEGs (need GPU decoding, e.g. nvJPEG), small files, unsupported hardware.
- Verdict: worth it for data-centre-scale pipelines, not for everyday photo rotation.

Task 2

Ethical Implications of Scaling HPC for AI



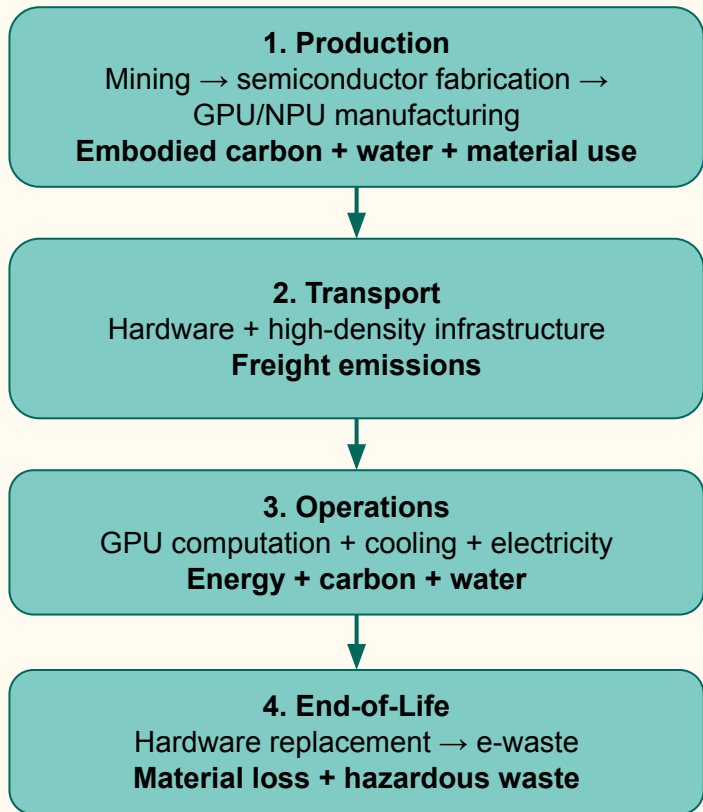
Ethical Implications of Scaling HPC for AI

Scaling HPC enables powerful AI systems, but also increases environmental impact, security and governance risks, and inequality in access to computational resources.

Environmental	Governance & Security
Energy, water & e-waste	Data privacy, security & bias
Socio-economic	Responsible HPC
Unequal access to compute	Power efficiency & equitable access

Responsible HPC must balance computational performance with environmental sustainability, security, fairness and accessibility.

Environmental & Energy Impact



Scaling creates resource pressure

AI compute is becoming a major electricity load

- U.S. data centres consumed **~4.4% of total electricity in 2023**
- Projected to reach **~13.2% by 2028** (LBNL projection)
- AI workloads also increase demand for **water-intensive cooling**

The benefits of faster AI innovation are accompanied by environmental costs that may be concentrated in the communities supplying electricity, water and infrastructure.

(Kandemir, 2025;
Parliament of Australia, 2024)

$$\text{PUE} = \frac{\text{Total Facility Energy}}{\text{IT Energy}}$$

$$\text{CUE} = \frac{\text{CO}_2\text{e Emissions}}{\text{IT Energy}}$$

$$\text{WUE} = \frac{\text{Facility Water Use}}{\text{IT Energy}}$$

Data Governance, Security & Fairness

Shared HPC → Massive distributed data → Parallel model training → Security + privacy + bias risks → Need for stronger governance

Shared HPC creates security risks

Multi-tenant GPU clusters

- Multiple users/workloads share physical infrastructure
- Shared GPU memory, PCIe buses and interconnect fabrics
- Isolation depends on scheduler and system-level controls
- Poor isolation can expose sensitive data or model information

Risk: Cross-tenant leakage + side-channel attacks

Distributed AI data creates governance challenges

Petabyte-scale datasets

- Distributed file systems enable high-throughput parallel I/O
- Training data may contain:
 - PII
 - copyrighted material
 - proprietary information
 - biased or unreliable content
- Data provenance and access controls become harder to audit at scale

Risk: Loss of privacy, ownership and accountability

Bias can scale with compute

Biased data → parallel training → model weights → biased decisions

More computational power can amplify the impact of biased or poorly governed data.

Ethical principle:

Compute scalability must be matched by governance scalability.

(OECD, 2022; Zhao et al., 2024)

The Compute Divide: Who Gets Access to AI?

Large technology companies	Universities & smaller researchers
Multi-billion-dollar infrastructure budgets	Limited research funding
Large-scale GPU/TPU clusters	Smaller or older clusters
Rapid foundation-model training	Longer or infeasible training
Proprietary infrastructure	Reliance on shared HPC/cloud access

Limited compute → limited research capability

- Researchers may be unable to train large models from scratch
- Independent **safety, bias and security audits** become harder
- Research agendas can shift towards what is computationally affordable
- Advanced AI capability becomes concentrated among organisations with the greatest resources

If computational power determines who can participate in AI research, access to compute becomes an issue of research equity. *(Ahmed & Wahed, 2020; Kandemir, 2025)*

Responsible & Sustainable HPC

1. Reduce energy & environmental impact

GPU power capping

- Use Slurm to enforce per-job GPU power limits
- Lower power and thermal load
- Trade-off: slightly longer execution time

MIT Supercloud study:

60% power cap → ~10 W lower average GPU power

~4.8°C lower GPU temperature

(Zhao et al., 2024)

2. Optimise the workload

Green AI

- Model pruning & sparsification
- Quantisation: **FP32** → **FP16/INT8**
- Knowledge distillation
- Smaller models where sufficient
- Measure energy/carbon during training

Goal:

Less computation → less energy → lower environmental impact

3. Govern compute fairly

Carbon-aware scheduling

Shift non-urgent workloads to:

- Lower-carbon electricity periods
- Regions with greater renewable availability

Equitable HPC access

- Public/shared HPC grants
- University compute allocations
- Support independent AI auditing

Conclusion: Scaling Responsibly

Scaling HPC enables major AI innovation, but also scales its consequences.

Challenge	Responsible response
Environmental impact	Energy & carbon efficiency
Security & governance	Strong data controls & auditing
Compute inequality	Equitable HPC access
Resource consumption	Power-aware & carbon-aware scheduling

The goal is not to stop scaling HPC, but to scale it responsibly.

References

Ahmed, N., & Wahed, M. (2020). The de-democratization of AI: Deep learning and the compute divide in artificial intelligence research. *arXiv*. <https://doi.org/10.48550/arXiv.2010.15581>

Kandemir, M. (2025, April 8). *Why AI uses so much energy—and what we can do about it*. Penn State Institute of Energy and the Environment. <https://iee.psu.edu/news/blog/why-ai-uses-so-much-energy-and-what-we-can-do-about-it>

OECD. (2022). *Measuring the environmental impacts of artificial intelligence compute and applications: The AI footprint* (OECD Digital Economy Papers No. 341). OECD Publishing.
https://www.oecd.org/en/publications/measuring-the-environmental-impacts-of-artificial-intelligence-compute-and-applications_7babf571-en.html

Parliament of Australia. (2024). *Chapter 6: Impacts of AI on the environment*. Select Committee on Adopting Artificial Intelligence.
https://www.aph.gov.au/Parliamentary_Business/Committees/Senate/Adopting_Artificial_Intelligence_AI/AdoptingAI/Report/Chapter_6_-_Impacts_of_AI_on_the_environment

Zhao, D., Samsi, S., McDonald, J., Li, B., Bestor, D., Jones, M., Tiwari, D., & Gadepally, V. (2023). Sustainable supercomputing for AI: GPU power capping at HPC scale. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (pp. 588–596). Association for Computing Machinery. <https://arxiv.org/html/2402.18593v1>